

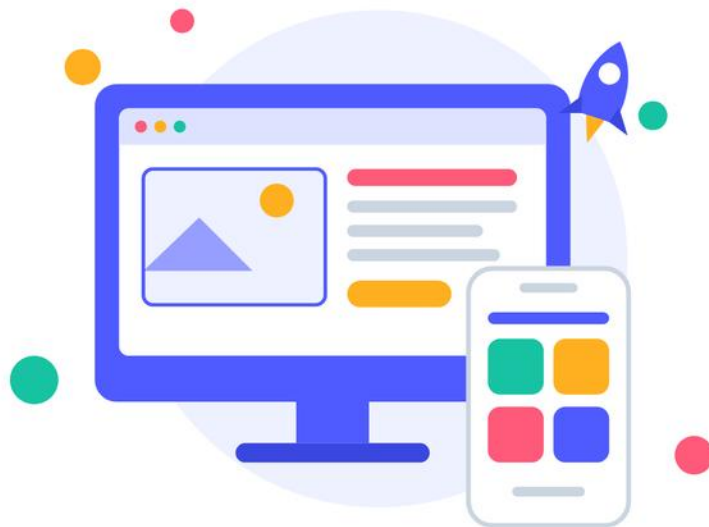
## WEEK 01 · REFERENCE

# THE HTML CATALOGUE

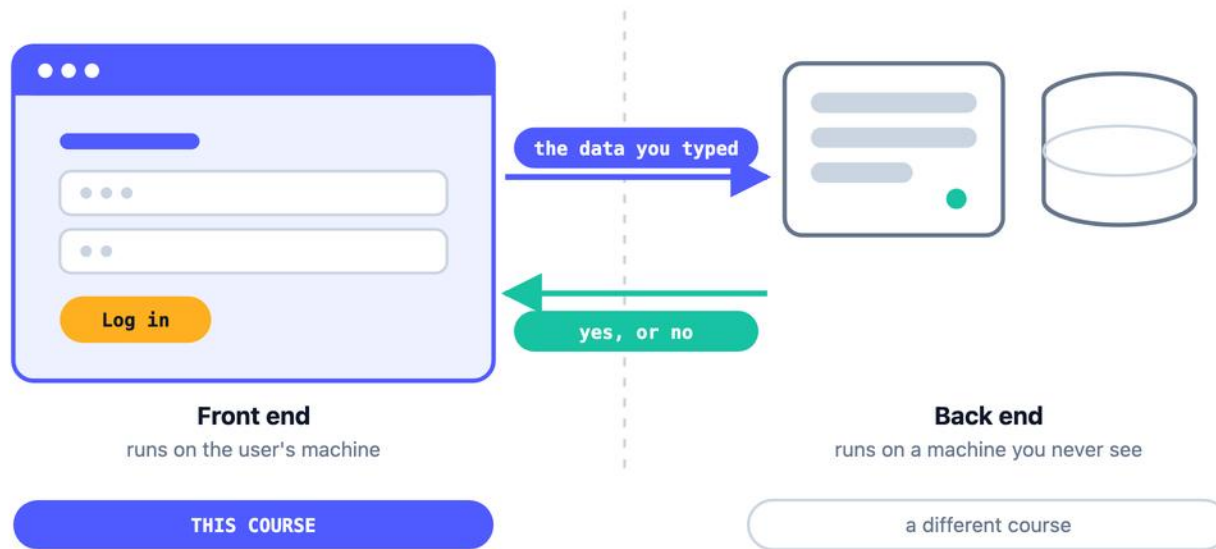
The companion to the spine, and the one you keep open. The catalogue, the six words in depth, every slide the cycles replaced, how the parser really builds the tree, and what git really stores. Optional depth, not examined — and still the copy you will want in week 7.

YOSSI ARVIV · FRONT-END WEB DEVELOPMENT · RUPPIN

*Everything graded this week was typed in a cycle. This deck is the copy you keep, and the place to go when you want to know why — not only how.*



# FRONT END, BACK END



## FRONT END — THIS COURSE

Everything that runs **in the browser, on the user's machine**. HTML, CSS, JavaScript. You can read all of it: press `Ctrl/Cmd+U` on any site and there it is.

## BACK END — NOT THIS COURSE

Runs **on the server**: databases, accounts, payments. You never see the code. It answers your requests and it does not trust you.

# DNS



DNS works like a phonebook. Without DNS, you'd have to memorize numbers instead of names.

`ruppin.ac.il` is for humans. A connection is opened to an **address**, so before anything else the browser has to ask someone what that address is.

► **TRY IT LIVE** — `ping ruppin.ac.il` in the terminal. One second, and the name becomes a number in front of the room.

# ONE TRIP, TWO MESSAGES



# THE S IS NOT DECORATION

## HTTP://

Every byte travels in the clear. Anyone between you and the server — the café wifi, the network you are on — can read it, and change it.

## HTTPS://

The same HTTP, inside TLS. Encrypted in transit, and the certificate proves the server is who it claims to be.



The padlock in the address bar means the connection is private — not that the site is honest. Both are worth saying out loud.

# REQUEST AND RESPONSE ARE JUST TEXT

## WHAT YOU SEND

```
1 GET /courses HTTP/1.1
2 Host: ruppun.ac.il
3 Accept: text/html
4 User-Agent: Mozilla/5.0 ...
```

## WHAT COMES BACK

```
1 HTTP/1.1 200 OK
2 Content-Type: text/html; charset=utf-8
3 Content-Length: 1841
4
5 <!doctype html>
6 <html lang="he">
7 ...
```

**GET** read · **POST** create · **PUT** replace · **DELETE** remove. **200** fine · **301** moved · **404** no such thing · **500** the server broke.

# MARKUP IN, TREE OUT

Type on the left – the DOM rebuilds on the right

↺ reload

open ↗

## 1 — THE TEXT YOU SEND

reset

Edit this. The tree on the right is rebuilt as you type.

```
<article>
  <h2>Trip report</h2>
  <p>We walked <strong>17 km</strong>.</p>
  
</article>
```

## 2 — THE TREE THE BROWSER BUILT

```
article Element
  h2 Element
    "Trip report" Text
  p Element
    "We walked" Text
    strong Element
      "17 km" Text
    "." Text
  img src="map.svg" alt="Route map" Element
```

Every node is an object with a parent, children and properties. You have written this data structure before — it is a tree, and you traversed one in Data Structures.

► **HANDS-ON** [demos/02-dom-tree/](#) — Type in the left pane. Then delete one `</p>` and watch the tree change shape.

# THE DOM IS A TREE. YOU HAVE WRITTEN ONE.

C#, SECOND YEAR

```
1 class Node {  
2     public string Tag;  
3     public Node Parent;  
4     public List<Node> Children;  
5 }  
6  
7 // walk it  
8 foreach (var c in node.Children) ...
```

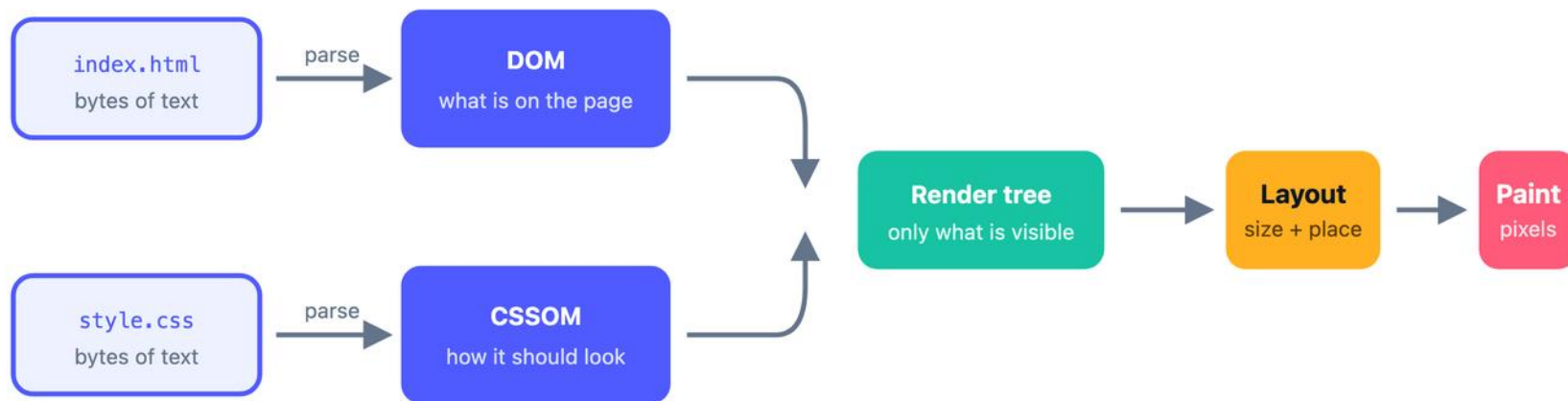
THE DOM, TODAY

```
1 element.tagName;    // "ARTICLE"  
2 element.parentNode; // the node above  
3 element.children;   // the nodes below  
4 element.textContent; // the text inside  
5  
6 // walk it  
7 for (const c of element.children) ...
```

The DOM is a tree-like structure created by the browser that represents the HTML elements as JavaScript objects.



# TWO TREES, THEN PIXELS



display:none never reaches the render tree — it has no box to lay out.

CSS blocks rendering on purpose. A page painted before its stylesheet arrived would flash unstyled, and you would call that a bug.

# DEVTOOLS SHOWS THE TREE, NOT YOUR FILE

## ELEMENTS

The *live* DOM. Edit here and the page changes; reload and it is gone.

## NETWORK

Every request, its status and its size. This is where 404s confess.

ELEMENTS IS NOT YOUR FILE

```
<!-- what you wrote -->  
<p>Trip report  
  
<!-- what Elements shows -->  
<p>Trip report</p>
```

**Six words for the whole trip:** request, DNS, TLS, response, parse, paint — and inside "paint": style, layout, composite.

You never typed that closing tag. The browser did, while building the tree — and the tree is what Elements is a window onto.

# HOW THE PARSER BUILDS THE TREE

- tokeniser** Characters in, tokens out: start tag, end tag, text, comment, doctype.
- tree builder** Each token becomes a node, inserted under the element currently open.
- the stack** A start tag pushes, an end tag pops. LIFO — the stack you already know.
- never stops** No fatal errors. Every mistake has one defined repair, the same in every browser.

WHAT AN UNCLOSED H1 DOES TO THE STACK

```
<main>
  <h1>Noa Levi
    <p>Third-year student.</p>
</main>

<!-- stack: html > body > main > h1 > p -->
<!-- the p is INSIDE the h1: 32px, bold -->
<!-- </main> pops everything down to main -->
```

Cycle 1 typed this. Elements showed the `p` as a child of the `h1`; the validator printed five lines for one missing tag.

# TAGS THE PARSER INSERTS FOR YOU

## WHAT YOU WROTE

```
<!doctype html>
<title>t</title>
<table><tr><td>x</td></tr></table>
<p>a
<div>b</div>
<ul><li>one<li>two</ul>
```

## WHAT ELEMENTS SHOWS

```
<html><head><title>t</title></head><body>
<table><tbody><tr><td>x</td></tr></tbody></table>
<p>a</p>
<div>b</div>
<ul><li>one</li><li>two</li></ul>
</body></html>
```

### implied start tags

`html`, `head`, `body` are created when you leave them out; `tbody` is inserted around a `tr` written straight into a `table`.

### implied end tags

A block element like `div` closes an open `p`; the next `li` closes the previous one; `</head>` is implied by the first body content.

### why you write them anyway

Because the repair is only predictable when you know the rules — and Prettier, the validator and week 9's `querySelector` all read what you wrote, not what the browser fixed.

# HTML IS NOT A PROGRAMMING LANGUAGE

HTML is the skeleton of every web page.

Not a programming language → it's a markup language.

There is no `if`, no loop, no variable. You are not telling the browser what to do. You are telling it **what things are**.

# THE SKELETON, LINE BY LINE

```
1 <!doctype html>
2 <html lang="en">
3   <head>
4     <meta charset="utf-8" />
5     <meta name="viewport"
6       content="width=device-width, initial-scale=1" />
7     <title>Maya Cohen</title>
8   </head>
9   <body>
10    <h1>Maya Cohen</h1>
11  </body>
12 </html>
```

# CHARSET — THE LINE THAT DECIDES IF HEBREW WORKS

## MISSING

```
1 <head>
2   <title>קורות חיים</title>
3 </head>
```

The browser guesses the encoding. On the wrong guess you get:

×§×•×"×•×@ ×-×™×™×

## DECLARED

```
1 <head>
2   <meta charset="utf-8" />
3   <title>קורות חיים</title>
4 </head>
```

Bytes are read as UTF-8. Hebrew, emoji and — all survive.

# VIEWPORT — THE LINE THAT DECIDES IF PHONES WORK

## MISSING

```
1 <head>
2   <meta charset="utf-8" />
3 </head>
```

The phone pretends to be a 980px desktop and shrinks the whole page. Text becomes unreadable and your media queries never fire.

## DECLARED

```
1 <head>
2   <meta charset="utf-8" />
3   <meta name="viewport"
4         content="width=device-width,
5               initial-scale=1" />
6 </head>
```

1 CSS pixel = 1 device-independent pixel. The page is laid out for the real screen.

# AN ELEMENT, NAMED

```
1 <a href="/pricing" class="btn">View pricing</a>  
2 <!-- ^tag ^attribute      ^content ^closing -->  
3 
```

- tag** the name — what this thing IS
- attribute** `name="value"` — extra information about it
- content** what sits between the tags
- void** elements with no content and no closing tag



# SOME ELEMENTS HAVE NOTHING TO CLOSE

A **void element** has no content, so it has no closing tag. Everything it says, it says in its attributes.

`</img>` and `</br>` are not "optional" — they are not a thing. Write one and the browser discards it, silently, like everything else.

```
1 
2 <br>
3 <hr>
4 <input type="email" name="email">
```

Four you will meet this week. Two of them — `br` and `hr` — you will meet again on the next slide, being used wrongly.

## VOID ELEMENTS — NO CONTENT, NO CLOSING TAG

```
  
<br>      <!-- a line break that is part of the content -->  
<hr>      <!-- a thematic break between sections -->  
<input type="email" id="email" name="email">  
<meta charset="utf-8">  
<link rel="stylesheet" href="style.css">
```

There is nothing to put inside them, so there is nothing to close.

`<img></img>` is not a thing. The living standard lists exactly thirteen:

`area` `base` `br` `col` `embed` `hr` `img` `input` `link` `meta` `source`  
`track` `wbr`. (`param` and `keygen` used to be on it; both are obsolete.)

# THE CHARACTERS YOU HAVE TO ESCAPE

## WHAT YOU MEANT TO SHOW

```
<p>Write <strong> for importance.</p>  
<p>Milk & bread</p>
```

## CHARACTER REFERENCES

```
<p>Write &lt;strong&gt; for importance.</p>  
<p>Milk &amp; bread</p>
```

The first one renders as bold text with no tag visible: the browser read `<strong>` as a tag, because it is one.

## the five

`&lt;`, `<`, `&gt;`, `>`, `&amp;`, `&`, `&quot;`, `"` (inside an attribute value) ·  
`&nbsp;`; a space that never breaks the line

## everything else

With `<meta charset="utf-8">` you type it: `—`, `©`, `ℕ`, Hebrew.  
`&#8212;` is the em dash spelled as a number; you will see it in old code, and you do not need it.

## the rule

Only `<` and `&` are ambiguous in text; only `"` is ambiguous inside a quoted attribute. Escape those, and nothing else.

# NESTING AND INDENTATION

## UNREADABLE

```
<main><section><h2>About</h2>  
<p>Text</p></section></main>
```

## TWO SPACES PER LEVEL

```
<main>  
  <section>  
    <h2>About</h2>  
    <p>Text</p>  
  </section>  
</main>
```

Both render identically. One of them you can still edit in week 12. Prettier does this on save — it is already configured in the folder you were given.

# ID AND CLASS

## ID – ONE, UNIQUE

Appears **once** per document. Targets an anchor ( `#contact` ), links a `label` to its field, and is what `getElementById` looks for.

## CLASS – MANY, REUSABLE

As many elements as you like, and an element can carry several. This is what CSS hangs on from week 3 onwards.

```
1 <section id="contact" class="panel panel--wide">...</section>
2
3 <a href="#contact">Jump to contact</a>
```

CYCLE 1 · WHAT IT BECOMES

# FROM AN EMPTY FILE

demos/01-first-page — this is where we arrive

↻ reload

open ↗

## Maya Cohen

Third-year student. Learning to build for the web.

- [About me](#)
- [Projects](#)

### About me

I already write C# and Python. This semester I am finding out what the browser actually does with the text I send it.

### Projects

- A page that validates clean — this one.
- Whatever the CSS week turns it into.

Built in class, from an empty file, in about eight minutes.

► **HANDS-ON** demos/01-first-page/ — Open 01-blank/ in Live Server FIRST — a white rectangle — then type the skeleton live.

This is the page cycle 1 arrives at, in a frame. Cycle 1 typed it from an empty file; live-code/01-empty-file-to-skeleton/end/ is the version you typed.

# THE WHOLE VOCABULARY, ON ONE PAGE

---

## DOCUMENT

`html` · `head` · `body` · `title` · `meta` ·  
`link` · `script` · `style`

---

## LISTS

`ul` · `ol` · `li` · `dl` · `dt` · `dd`

---

## FORMS

`form` · `fieldset` · `legend` · `label` ·  
`input` · `textarea` · `select` ·  
`option` · `button`

---

## SECTIONING

`header` · `nav` · `main` · `section` ·  
`article` · `aside` · `footer` · `h1–h6`

---

## LINKS AND MEDIA

`a` · `img` · `picture` · `source` ·  
`figure` · `figcaption` · `video` ·  
`audio` · `svg` · `iframe`

---

## INTERACTIVE

`details` · `summary` · `dialog` ·  
`progress` · `meter`

---

## TEXT

`p` · `strong` · `em` · `b` · `i` · `small` ·  
`mark` · `time` · `code` · `pre` ·  
`blockquote` · `abbr`

---

## TABLES

`table` · `caption` · `thead` · `tbody` ·  
`tfoot` · `tr` · `th` · `td`

---

## MEANINGLESS ON PURPOSE

`div` · `span` — and that is the entire point of  
them.

About 110 elements exist. These are the ones this course uses. Every one of them is on MDN, one search away — the value of this page is knowing what to search for.

# THE SEMANTIC ELEMENTS, ALL OF THEM

## PAGE REGIONS

header nav main footer  
aside

## CONTENT

section article figure  
figcaption time blockquote

## TEXT MEANING

strong em abbr code mark  
small

This content should be meaningful on its own, even if separated from the rest of the webpage.

## DIV AND SPAN MEAN NOTHING — ON PURPOSE

### <DIV> — A BLOCK

Takes the full width and starts a new line. A box you group things into.

### <SPAN> — INLINE

Sits inside a line of text without breaking it. For a few words, not a region.

```
1 <div class="price-box">
2   Total: <span class="currency">£</span>149
3 </div>
4
5 <!-- neither says what the content IS. That is the whole point. -->
```

Both are legitimate. Reach for them when there is genuinely no meaning to express — and reach for `section` or `article` when there is.

# THREE RULES THAT WILL SAVE YOU MARKS

Avoid using `<section>` purely for styling — use `<div>` for that.

Headings are not for styling text big/bold.

Link text should describe the destination ("View pricing"), not "Click here".

# H1 IS AN OUTLINE LEVEL, NOT A FONT SIZE

## CHOSEN BY SIZE

```
1 <h1>My site</h1>
2
3 <h4>About me</h4>
4 <p>...</p>
```

"h4 because h2 looked too big."

## CHOSEN BY STRUCTURE

```
1 <h1>My site</h1>
2
3 <h2>About me</h2>
4 <p>...</p>
5
6 /* in week 3, in CSS: */
7 h2 { font-size: 1.25rem; }
```

# <BR> IS NOT SPACING

## FAKED WITH BR

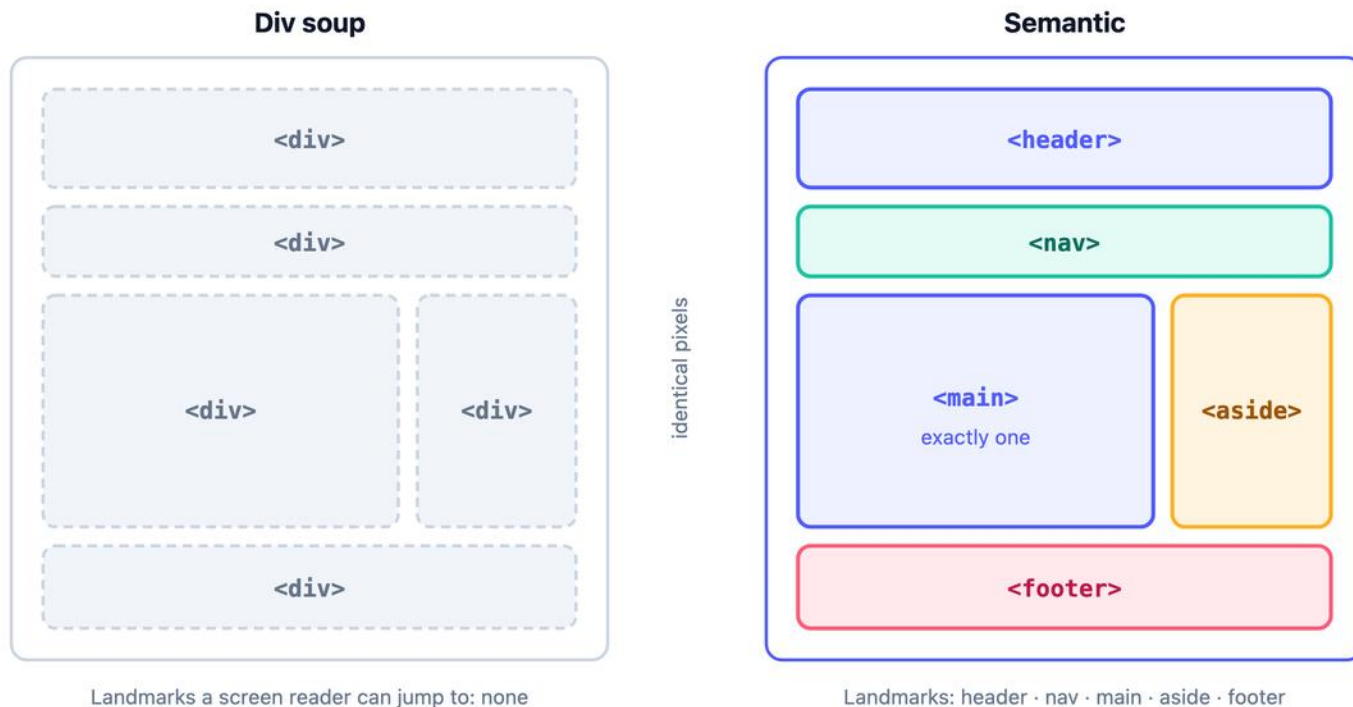
```
1 Opening hours<br>
2 <br>
3 Sunday to Thursday<br>
4 08:00–16:00
```

## REAL ELEMENTS

```
1 <h2>Opening hours</h2>
2 <p>
3   Sunday to Thursday<br />
4   08:00–16:00
5 </p>
```

`<br>` means "this line break is part of the content" — an address, a poem, a postal label. It is not a way to push things apart.

# SAME PIXELS, DIFFERENT KNOWLEDGE



# WHAT DIV SOUP ACTUALLY COSTS YOU

- Week 6** Tailwind on a div soup produces class strings nobody can read or change.
- Week 9** `querySelector('main')` returns `null`. There is no main.
- Week 13** In the exam you get 90 minutes on your own project. You will navigate it by structure, under time pressure, with no search engine.
- Every week** Structure is graded from today and accessibility from HTML II — not from week 7.

# LISTS — THREE KINDS, AND THEY NEST

```
1 <ul>      <!-- order does not matter -->
2   <li>Milk</li>
3   <li>Bread</li>
4 </ul>
5
6 <ol>      <!-- order IS the meaning -->
7   <li>Install Node</li>
8   <li>Then clone</li>
9 </ol>
10
11 <dl>      <!-- term and description -->
12   <dt>DNS</dt>
13   <dd>A name becomes an address</dd>
14 </dl>
```

## NEVER

Do not number an `<ol>` by hand — you get 1. 1. 1. the moment you reorder. And never fake a list with `<br>`.

## ONLY LI

The only legal direct child of `ul / ol` is `li`. Put anything else inside an `li`.

CYCLE 2 · SELF-STUDY

# BREAK IT THREE WAYS

demos/06-validator/01-broken – three planted errors

↶ reload

open ↗

## Trip report — Golan, March

### The route

Seventeen kilometres, most of it uphill, all of it in the rain.

### The photographs



[The version that validates](#)

► **HANDS-ON** demos/06-validator/ — Run `npx html-validate@9` on this file in the integrated terminal. Do not say what is broken.

# WHAT THE VALIDATOR'S `close-order` CASCADE MEANS

```
1 72:8 error Unclosed element '<section>' close-order
2 79:8 error Stray end tag '</div>' close-order
3 81:20 error Duplicate ID "hours" no-dup-id
4 86:12 error <p> element is not permitted as content under <ul> element-permitted-content
5 91:6 error End tag '</main>' seen but there were open elements close-order
6 94:4 error End tag '</body>' seen but there were open elements close-order
7 95:2 error End tag '</html>' seen but there were open elements close-order
```

## lines 1–2 · one mistake

A `</div>` where `</section>` belonged. The section stays on the stack; the div was never opened.

## lines 5–7 · the same mistake

Every end tag after it finds the section still open and reports it. Fix line 2 and these three vanish with it — five lines from one character.

## lines 3–4 · two more mistakes

Independent, one line each: a duplicate id, and a `p` as a direct child of a `ul`. They stay until you fix them.

## the rule

Read the first line. Fix it. Run again. Never try to fix seven lines at once — the count is not the number of mistakes.

# ANCHORS — EVERY KIND OF LINK

```
1 <a href="about.html">Another page in this folder</a>
2 <a href="../index.html">Up one folder</a>
3 <a href="#contact">A section of this page</a>
4 <a href="mailto:me@ruppin.ac.il">Open a mail client</a>
5 <a href="https://mdn.io" target="_blank" rel="noopener noreferrer">A new tab</a>
```

## THE ONE RULE

The link text must make sense read on its own. A screen-reader user can ask for a list of every link on the page — ten of them saying “Click here” is a list of nothing.

# FIVE KINDS OF HREF

```
1 <a href="https://developer.mozilla.org">MDN</a>
2 <a href="about.html">The longer version</a>
3 <a href="#contact">Jump to contact</a>
4 <a href="mailto:maya@ruppin.ac.il">Email me</a>
5 <a href="tel:+972501234567">Call me</a>
```

## absolute

Another site. Full scheme and host, and it never changes meaning.

## relative

A file next to this one. **The only kind that survives being moved to my machine** — this is the paths section, three slides from now.

## #fragment

An `id` on this page. `href="#contact"` finds `id="contact"`. This is what your `nav` uses.

## mailto:

Opens a mail client with the address filled in.

## tel:

On a phone, dials. On a laptop, usually nothing — and that is fine.



# THE href CATALOGUE

```
1 <a href="about.html">same folder</a>
2 <a href="../index.html">one folder up – counted from THIS file</a>
3 <a href="#contact">an id on this page; never leaves the browser</a>
4 <a href="search.html?q=coffee&page=2">a query string; reaches the server</a>
5 <a href="https://developer.mozilla.org/">absolute: scheme + host, never changes meaning</a>
6 <a href="//cdn.example.com/lib.js">protocol-relative: inherits http or https from the page</a>
7 <a href="mailto:me@ruppin.ac.il">a mail client</a> · <a href="tel:+972501234567">a dialler</a>
8 <a href="cv.pdf" download>save instead of navigate (same-origin files)</a>
```

**what travels** Relative paths and absolute URLs. A leading slash is "from the server root", which is a different folder on every server; a drive letter or `file://` is one machine.

**the fragment** `#contact` is resolved by the browser against `id="contact"`. It is not sent in the request — the Network tab never shows it.

**the query** Everything after `?` goes to the server as `name=value` pairs joined by `&`. A form with `method="get"` builds exactly this string — next week.

# THE THREE BROKEN PATHS, IN A FRAME

demos/04-paths/01-broken – valid HTML, three 404s

reload

open

## Three paths, three ways to fail

1. `file:///Users/maya/Desktop/site/images/logo.svg`



An absolute path into someone's Desktop. Nobody else has that folder.

2. `C:\Users\maya\Desktop\site\images\logo.svg`



Same mistake, Windows flavour. Backslashes are not path separators on the web.

3. `/images/logo.svg`



► **HANDS-ON** `demos/04-paths/` — Open DevTools > Network and reload. Three 404s. Then walk `02-fixed/` both ways.

# A PAGE IS ALWAYS OPENED WITH LIVE SERVER

DOUBLE-CLICK

```
file:///Users/maya/Desktop/site/index.html
```

LIVE SERVER

```
http://127.0.0.1:5500/index.html
```

**Week 8** ES modules refuse to load over `file://`.

**Week 11** `localStorage` is scoped to an origin. `file://` barely has one.

**Week 11** `fetch` is blocked outright.

## figure, figcaption — AND time, NEXT WEEK

AN IMAGE WITH A CAPTION THAT BELONGS TO IT

```
1 <figure>
2   
4   <figcaption>The summit at 09:40, before the rain.</figcaption>
5 </figure>
```

A DATE A MACHINE CAN READ — HTML II

```
1 <p>Published <time datetime="2027-11-02">2 November 2027</time>.
```

`alt` says what is in the picture; `figcaption` says why this exists for the reader; `datetime` is for the machine, in ISO 8601 format (year-month-day). They are different sentences, and a screen reader reads both. Not graded this week — it is next week's material, listed here so the catalogue is complete.

HTML II · NEXT WEEK

# CLICK THE WORDS "FULL NAME"

demos/05-forms/01-broken — nothing happens

↻ reload

open ↗

## Contact us

Full name

Email

Message

send

Press "send" to see what the server would receive.

► **HANDS-ON** `week-02-html-forms-a11y/demos/05-forms/` — Click the words “Full name”. Nothing. Press `send`. Nothing. Then run the validator on it — it passes.

# THE FORM, DONE PROPERLY

```
1 <form action="/contact" method="post">
2   <fieldset>
3     <legend>Your details</legend>
4
5     <label for="name">Full name</label>
6     <input type="text" id="name" name="name" required />
7
8     <label for="email">Email</label>
9     <input type="email" id="email" name="email" required />
10
11    <button type="submit">Send</button>
12  </fieldset>
13 </form>
```

# PLACEHOLDER IS NOT A LABEL

## PLACEHOLDER ONLY

```
1 <input type="text" placeholder="Email" />
```

Disappears the moment you type. Fails contrast.  
Cannot be clicked.

## LABEL + FOR

```
1 <label for="email">Email</label>  
2 <input type="email" id="email" name="email" />
```

`for` points at `id`. Clicking the label focuses the field.

placeholder is not a label, always provide a real `<label>`.

# TYPES DO WORK FOR YOU. NAME DECIDES WHAT IS SENT.

## type

email tel number date url

password checkbox radio — each

brings its own keyboard on mobile and its own validation.

## name

The key the server receives. No name, no data — the field is ignored.

## id

Unique in the document. What label for points at.

## required

The browser refuses to submit an empty field, and says so in the user's language.

WHAT THE SERVER ACTUALLY RECEIVES

```
<input type="email" name="email" id="a" required>  
<input type="text" name="name" id="b">  
<input type="date" id="c">
```

?

email=maya%40ruppin.ac.il&name=May

a

The third field has no name. It is on the page, the user filled it in, and it is not in that string.

# FORMS — THE INPUT TYPES WORTH KNOWING

|                 |                                                              |
|-----------------|--------------------------------------------------------------|
| <b>text</b>     | anything short                                               |
| <b>email</b>    | validates the shape, and changes the phone keyboard          |
| <b>tel</b>      | numeric keypad on mobile                                     |
| <b>number</b>   | with <code>min</code> , <code>max</code> , <code>step</code> |
| <b>date</b>     | a real date picker, free                                     |
| <b>password</b> | masked — masked, not secure                                  |
| <b>checkbox</b> | many independent yes/no                                      |
| <b>radio</b>    | one of several — same <code>name</code>                      |
| <b>file</b>     | upload                                                       |
| <b>textarea</b> | multi-line; not an input                                     |
| <b>select</b>   | a list of <code>option</code> s                              |

Create Account

Full name

Email

Newsletter topics

☐ Tech

☐ Deals

Plan

☒ Basic

☐ Pro

Country

About you

Every field here has a real label. That is the assignment.

# TABLES — THE FULL ANATOMY

```
1 <table>
2   <caption>Opening hours</caption>
3   <thead>
4     <tr>
5       <th scope="col">Day</th>
6       <th scope="col">Hours</th>
7     </tr>
8   </thead>
9   <tbody>
10    <tr>
11      <th scope="row">Sunday</th>
12      <td>08:00–16:00</td>
13    </tr>
14  </tbody>
15 </table>
```

**scope** is the whole point: it tells a screen reader which header owns which cell. A table without it is a grid of unlabelled numbers.



# ACCESSIBILITY IS GRADED FROM THIS WEEK — AXE FROM NEXT WEEK

## Keyboard

Tab through the whole page. Everything interactive must be reachable.

## Focus

You must be able to *see* where you are. Never delete the focus ring.

## Landmarks

`header` `nav` `main` `footer` — how a screen-reader user skips to the content.

## Names

Every image, every field, every link says what it is.

Every assignment from HTML II onwards has a line for this. This week grades the landmarks, the alt and the nav list — the first axe run is next week.

REFERENCE · GIT

# YOU ALREADY HAVE A VERSION CONTROL SYSTEM

```
index.html  
index-v2.html  
index-final.html  
index-final-FINAL.html  
index-final-FINAL-use-this-one.html
```

It is bad. Which one did you hand in, and what exactly changed between two of them?



# GIT STORES SNAPSHOTS, NOT DIFFERENCES

## WHAT PEOPLE ASSUME

Git saves a list of changes and replays them to rebuild a version.

## WHAT GIT DOES

Every commit is a complete picture of the project at that moment. Unchanged files are stored once and pointed at.

This is why checking out an old commit is instant, and why nothing is ever "reconstructed". The diff you read is *computed* for you, not stored.

# WHAT GIT ACTUALLY STORES: BLOB, TREE, COMMIT

## blob

One file's content — not its name, not its date. Identical files anywhere in history are one blob.

## tree

A folder: names pointing at blobs or other trees. One snapshot of the project is one tree.

## commit

Points at one tree and at its parent, and carries the author, the date and the message.

## the SHA

Every object is named by the hash of its content — forty hex characters. Change one byte and the name changes: that is why a SHA freezes a submission.

```
git log -1 --format=%H
9f4c1a7e0b2d5836a1c4f7e9d2b8a6c3e5f10947

git cat-file -p HEAD          # tree, parent, message
git cat-file -p HEAD^{tree}  # the folder itself
```

Snapshots, not differences: a commit is a whole tree. The diff you read is computed from two trees, never stored.

# THE FOUR COMMANDS YOU WILL ACTUALLY RUN

## WHEN YOU ARE LOST

`git status` always answers. Run it before and after everything else.

```
1 git init
2
3 git status
4
5 git add index.html
6
7 git commit -m "feat: add semantic page skeleton"
8
9 git log --oneline
```

Staging exists so one messy afternoon can become three clean commits.  
Use it.

# THREE FILES THAT SHIP WITH EVERY PROJECT

## README.md

What this is and how to run it. The first thing anyone opens — including me.

## .gitignore

What must never be committed:

`node_modules/`, `.env`, `.DS_Store`.

Secrets in a public history stay there forever.

## .gitattributes

\* `text=auto eol=lf`. Already in your repo. It normalises line endings so a Mac and a Windows machine produce the same commit.

### WITHOUT .GITATTRIBUTES

```
- <h1>Maya Cohen</h1>
- <p>Third-year student.</p>
+ <h1>Maya Cohen</h1>
+ <p>Third-year student.</p>
```

Two identical lines, reported as changed, because one machine ended them with `\r\n` and the other with `\n`. Do this across a whole file and nobody can review anything.

## `.gitignore` PATTERNS — THE ONES IN YOUR TEMPLATE

```
1 node_modules/      # a FOLDER of this name, anywhere in the repo
2 .env .env.*        # a file of this name, anywhere – and every .env.something
3 *.log *.swp *~     # every file ending in .log; editor scratch files
4 .DS_Store          # the macOS folder-view file – why cycle 5 never saw it (also Thumbs.db, desktop.ini)
5 .idea/ dist/       # an editor's folder; the week-12 build output (dist/, a folder, anywhere)
6 !keep.log          # NOT in your template: ! un-ignores a file a pattern above caught
```

### where it lives

At the repository root, committed, so every clone ignores the same things. The template ships one with the eleven patterns above (and `*.local`, `*.tsbuildinfo`, `npm-debug.log*`).

### what it cannot do

Un-track a file already committed: `git rm --cached <file>` first. A secret in history stays in history.

### what it does not cover

Your own backup copies — `index-old.html` is a name no pattern knows. `git status` before `git add` is the only protection.

# COMMIT MESSAGE PREFIXES — THE CONVENTION THIS COURSE READS

|                  |                                                                                   |
|------------------|-----------------------------------------------------------------------------------|
| <b>feat:</b>     | something the page can do now that it could not — a section, a nav, a second page |
| <b>fix:</b>      | something that was wrong and is now right — a path, an alt, an unclosed tag       |
| <b>docs:</b>     | README, comments, prose                                                           |
| <b>style:</b>    | formatting only — Prettier ran                                                    |
| <b>refactor:</b> | the same page, restructured                                                       |
| <b>chore:</b>    | housekeeping — <code>.gitignore</code> , configuration                            |

Then one imperative line, about 50 characters, that says *what changed*:

```
feat: add the main navigation and two sections. Not update,  
not asdf.
```



# WHERE TO LOOK THINGS UP

## MDN

`developer.mozilla.org` — the reference. Search “mdn <tag>” and you land on the right page.

## THE VALIDATOR

`validator.w3.org`, or the checks already in your repo.

## NOT A RANDOM BLOG

Half of them are ten years old and still teaching `<center>`.